

Primer Parcial ICS

Temas

Unidad 1 completa (incluye paper No silver bullet)

- Introducción a la Ingeniería del Software. ¿Qué es?
- Estado Actual y Antecedentes. La Crisis del Software.
- Disciplinas que conforman la Ingeniería de Software.
- Ejemplos de grandes proyectos de software fallidos y exitosos.
- Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.
- Procesos de Desarrollo Empíricos vs. Definidos.
- Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software
- Ventajas y desventajas de c/u de los ciclos de vida.
- Criterios para elección de ciclos de vida en función de las necesidades del proyecto y las características del producto.
- Componentes de un Proyecto de Sistemas de Información.
- Vinculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

Unidad 2

- Gestión de Producto
- Requerimientos Ágiles
- User Stories
- Estimaciones Ágiles
- Framework Scrum

Unidad 3

- SCM

Unidad 1

Introducción a la Ingeniería de Software:

Software → set de programas y la documentación que lo acompaña

Tipos:

- System Software
- Utilitario
- Application Software

Software no se puede comprar con la manufactura

→ Ningún producto de software es igual a otro: x q los requerimientos van a ser diferentes

→ El software no es predecible, es intangible

→ El software no se gasta, puede dejar de tener vigencia pero no se desgasta en termino de material

Historia:

1968 → Nace el termino del software

El mítico hombre mes → plantea los errores en la gestión tradicional para el desarrollo de software y algunos de esos errores todavía siguen pasando hoy en día

Paradigma analisis estructurado

No silver bullet → características esenciales del software, no hay una manera única de resolver todas las problemáticas en la creación del software

CMM → modelos de calidad de software

2000 → manifiesto ágil

Problemas en el desarrollo de software

- Mas tiempos y costos que los presupuestados
- Mala calidad
- Mas documentación
- No es facil extenderlo y/o adaptarlo. Agregar en otra versión es casi una misión imposible
- La versión final del producto no satisface las necesidades del cliente

Aspectos claves para el exito:

- Involucramiento del usuario
- Apoyo de la gerencia
- Enunciado claro de los requerimientos
- Planeamiento adecuado
- Expectativas realistas
- Hitos intermedios
- Personas involucradas competentes

3 ramas:

- Disciplinas Técnicas
- Disciplinas de Gestión
- Disciplinas de Soporte

80% → costo de persona

10 % → costo conectividad

10 % → costo extras

Que tipo de personas → con habilidades y motivada

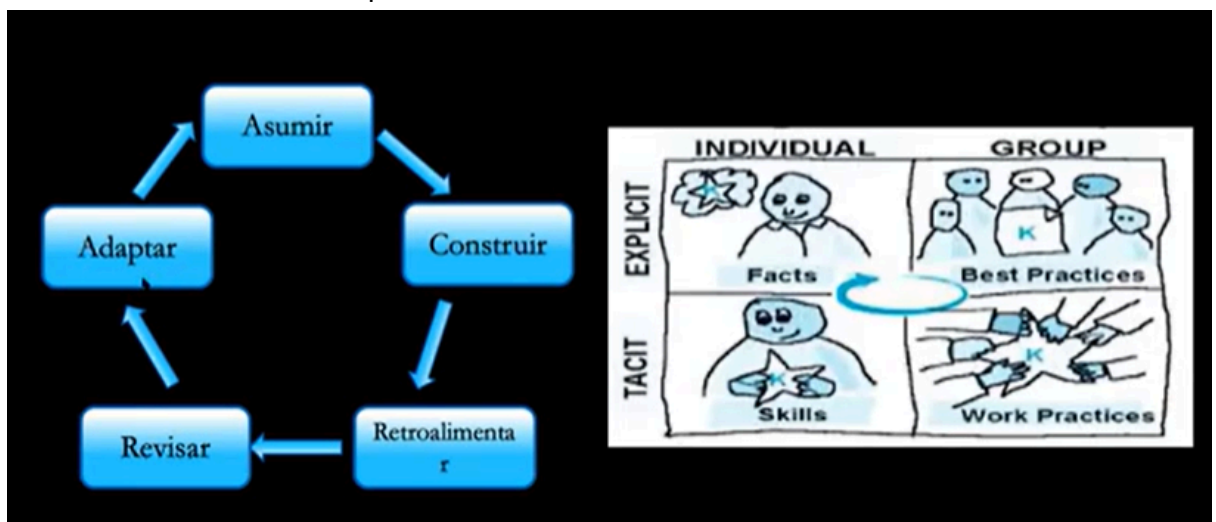
Herramientas de soporte

Procedimientos y métodos

Proceso: definición de un conjunto estructurado de actividades para desarrollar un producto de software (un objetivo) → definición teórica de lo que hay que hacer



- Definidos: EJ PUD. Definido paso a paso que es lo hay que hacer. Intención de ser repetitivos para poder predecir y tener visibilidad saber que va a pasar, ilusión de control. Intentan ser procesos completos. Cree que la experiencia se puede aplicar a cualquier equipo. Basados en el modelo industrial. Asumimos que si repetimos el mismo proceso vamos a lograr el mismo resultado
- Empíricos: fuerte arraigo en la experiencia, basarse en algo anterior pero esta se genera no se nace con experiencia. Procesos no completos, son guías, lineamientos, heurísticas que cubren algunos aspectos pero no se cubre la totalidad de las cosas que hay que hacer x hay partes que se deciden en base a la experiencia. La experiencia se gana sobre la retroalimentación, pero la experiencia de ese equipo solo sirve para ese equipo. Las variables no pueden ser controladas en su totalidad y la repetición de un mismo proceso puede tener distintos resultados. Personas → la clave del empirismo



Proyecto: Unidad de gestión, medio x el cual yo administro los recursos que necesito y las personas que requiero para desarrollar un producto de software. Incorpora personas que van a asumir roles que estan establecidos en el proceso (1 persona puede asumir mas de un rol, y un rol puede ser asumido por mas de una persona). Instanciamos el proceso para tener el proyecto. El proyecto es un medio para obtener una versión del producto. Temporalidad.

Características del proyecto → 4

- El producto desarrollado es único → x q cada versión resultante de un proyecto es distinta
- Objetivos claros y alcanzables
- Los objetivos guían al proyecto
- Duración Limitada → el proyecto inicia y termina (liberamos los recursos)
- Tareas interrelacionadas basadas en recursos → las tareas que tenemos que hacer las sacamos del proceso

Los proyectos se planifican ? → SI El acto de planificar es lo que sirve, un plan no sirve de nada si no lo pensé. Dejar registro de las tomas de decisiones para que todos sepamos que estamos haciendo y todos hayamos entendido lo mismo. Lo importante es lo que se aprende en el proyecto.

Administración del proyecto:

- Organizo los recursos
- Planificación del proyecto
- Seguimiento y monitoreo → para que todo vaya como yo quiero

Tradicional:

Gestión del proyecto → Lider de proyecto

Salida → Plan de proyecto: alcances, proceso, ciclo de vida, estimación, riesgos, recursos, programación de proyectos, controles, métricas

Lograr un equilibrio entre las 3 dimensiones del triangulo de hierro → 3 restricciones

- Alcances
- Costo
- Tiempo

Aunque sea 2 de estas dimensiones las debemos poder controlar. La que nunca depende de nosotros → LOS ALCANCES

Si esto se desequilibra se pone en riesgo en la CALIDAD.

Producto: El software. Conocimiento empaquetado en distintos niveles de abstracción, todo lo que se genera de el desarrollo (entregables) del producto es software. Todo lo que sale como resultado de la ejecución de las tareas es un producto de software. NO ES SOLO EL CÓDIGO.

Definición del alcance:

- Alcance del producto → ERS (Especificación de requerimientos de software)
 - características que deben incluirse en un producto o servicio
- Alcance del proyecto → PLAN DE PROYECTO
 - todo el trabajo y solo el trabajo que debe hacerse para desarrollar software (el producto)

Ciclos de vida

Abstracción, cuales son las etapas y cual es el orden

Fases y el orden de las fases → pero no dice quien lo va a hacer ni nada de eso

No es lo mismo que un proceso

Un proceso decide el paso a paso lo que tenemos que hacer → que hay que hacer en cada tarea

Es la representación de un proceso

Un ciclo de vida explica como vas a organizar y gestionar esas actividades

El mismo proceso que se puede usar con diferentes ciclo de vida

Hay ciclos de vida para los productos (inicia con la idea de crear un producto de software y se terminan cuando salen del mercado, mas largos y dentro de estos puede haber n ciclos de vida de proyecto) y ciclos de vida para los proyectos

Clasificación:

- Secuencial
 - Cascada
- Iterativo/Incremental → repite y avanza (incrementos al mismo producto)
- Recursivo → permiten atacar problemáticas específicas
 - Espiral

En un proceso definido se puede usar cualquier ciclo de vida

En un proceso empírico solo recomienda el Interactivo/Incremental → NUNCA SCRUM CON CASCADA



4 dimensiones del desarrollo de software:

80% proyectos de software fallan en la licitación de los requerimientos

Proceso mas abstracto y lo debo de adaptar en un proyecto y lo pienso en Roles → q voy a necesitar y dps ese proceso se instancia en el proyecto

Proyecto → unidad organizativa y necesita una indicación de como realizar el trabajo y el proceso es el que da una definición teórica de como llevar a cabo un desarrollo de software y el proyecto toma de ese proceso lo necesario para hacer y toma esas partes necesarias para hacer que van a formar parte del alcance.

Unidad 2

Filosofía ágil/manifiesto ágil:

→ Sustentada en los procesos empíricos

- Importante → ciclos de realimentación cortos

- Tiempo definido → al final de una iteración tengo que tener algo para entregar y si no tengo todo entrego menos

- Movimiento que se gestiona desde los desarrolladores (referentes de la industria de software)

- Es un compromiso, la forma de trabajar de una determinada manera

Empirismo

Pilares:

- Transparencia
- Inspección
- Adaptación

Manifiesto ágil:

4 valores

- Individuos e interacciones por sobre procesos y herramientas: es mas importante el vinculo y la comunicación que se establece que las aferrarse a las herramientas y a los procesos que vamos a hacer
- Software funcionando por sobre documentación extensiva → NO SIGNIFICA QUE NO SE VA A DOCUMENTAR
- Colaborar con el cliente por sobre negociacion contractual → Cambios en los requerimientos (este es el problema mas grande donde empiezan los ptos de fricción x q lo que esta firmado cambia), depende tmb del cliente en un contexto hostil es difícil ser ágil
- Responder al cambio por sobre seguir un plan : mantener una actitud abierta al cambio, para poder responder de una forma mas flexible y mas acertada a la realidad → nos ayuda a tener éxito

12 principios del manifiesto ágil:

- Principio 1 Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor
- Principio 2 Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente

- Principio 4 Los responsables del negocio y los desarrolladores trabajaremos juntos de forma cotidiana durante todo el proyecto
- Principio 6 El método mas eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara
- Principio 9 Atención continua a la excelencia técnica y a las bases de diseño: la calidad del producto no se negocia
- Principio 10 La simplicidad es esencial: lo mejor es lo simple quedarme con lo que el cliente me pidió
- Principio 11 Las mejores arquitectura, requisitos y diseño emergen de equipos auto-organizados

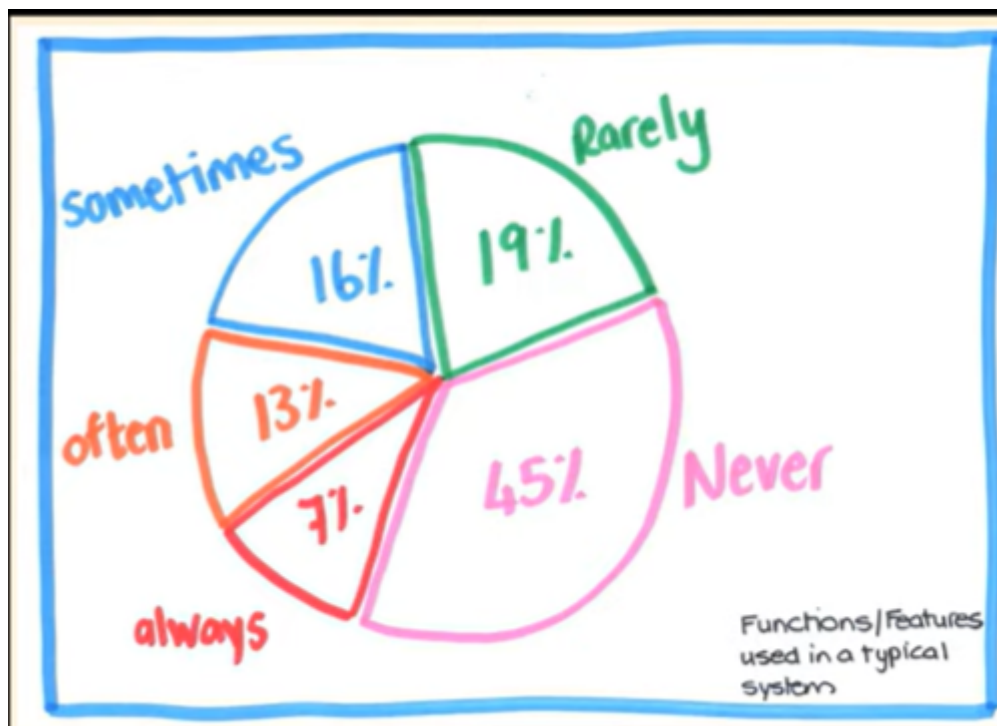
Requerimientos Ágiles → USER STORIES

Pilar:

→ Construir un Valor de Negocio

→ Vamos a ir encontrando los requerimientos de a poco (empezamos con solo lo suficiente, lo minimo necesario y dpps vamos agregando)

x q hay que cambiar a este enfoque:



La mayor cantidad de funcionalidades que se entregan al cliente no se usan. X eso se intente contrarrestar este situación

→ Tenemos un Product Owner que conoce las necesidades del negocio y es el que decide lo que se necesita en el producto

- Trabaja en un product backlog que es una pila o cola de funcionalidades, y el se encarga de priorizar esos requerimientos



Tenemos que tener a alguien que pueda tomar decisiones sobre el producto y que este dispuesto a trabajar con el equipo para lograr el mejor resultado.
 El producto no va a estar especificado desde el comienzo completo sino just in time de a poco acorde a cuando las necesitamos van a aparecer los requerimientos
 Tenemos que entregarle al cliente un valor de negocio

En el enfoque ágil cambian las prioridades:



Nuestro triangulo de hierro nos dice que tengo el tiempo fijo y los recursos fijos entonces con e tiempo y el equipo que tengo cuantos alcances o cuantos requerimientos puedo desarrollar

→ Hay que cambiar la cabeza del cliente para que entienda que no vamos a saber desde el inicio todo lo que va a abarcar el proyecto

Las user stories no trabajan en requerimientos funcionales (de software) sino en requerimientos de usuarios.

Trabajar juntos entendiendo las necesidades del negocio → Nos convertimos en expertos del dominio y descubrir con el usuario la mejor forma para satisfacer las necesidades

- Los cambios son la única cte
- Stakeholders: no son todos los que estan -> hay uno que toma las decisiones sobre las decisiones que e van a llevar a cabo
- Siempre se cumple eso de que “El usuario dice que quiere cuando recibe lo que pidio”
- No hay técnicas ni herramientas que sirvan para todos los casos
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar un resultado, una solución de valor



User Stories

Se llama historia de usuario: descripción corta de una necesidad que tiene el usuario sobre el producto de software

Partes de una US:

- Tarjeta
- Conversación → La parte mas importante y no queda guardada completa sino queda guardad en la parte visible de una US la tarjeta
- Confirmación → PU que se identifican necesarias para que el usuario me acepte la US

Forma de expresar las US:

Como <nombre del rol> yo puedo/quiero <actividad> de forma tal que/para <valor de negocio que recibo>

Como “usuario” → esta mal; debe ser UN ROL quien lo va a utilizar, SALVO en la US de loguearse

El valor de negocio le sirve para priorizar en el PB al PO

Frase Verbal → Forma corta para referenciar la US (hace foco en una manera corta para referirse a una US pero no puede ir sola)

Son multipropósito:

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

Las US son porciones verticales



Tengo que entregarle al usuario un poco de todo un poco de interfaz de usuario un poco de lógica y un poco de la BD. X q horizontalmente no le entrego ningun tipo de valor de negocio al Usuario

Modelado de Roles: Tarjeta de Rol de usuario

→ describe el rol (quien va a usar el sistema, si le va a ser útil o no, etc)

Cuando no tenemos PO (Proxies):

- Gerentes de Usuario

- Vendedores
- Expertos de dominio
- Clientes

TIENEN QUE SER PARTE DEL NEGOCIO NO TÉCNICOS

Criterios de Aceptación de las US:

Información concreta que nos dice si lo que implementamos es correcto, es decir si será aceptado o no

- Define límites para la US
- Son la base para la construcción de las PU

→ Solo acepto la US si cumple con los CA

Se escriben a nivel de usuario

Debe ser objetivo, claro y específico

Los detalles

- cada equipo decide donde poner o guardar los detalles
- que se ponen ahora en el diseño de pruebas → pruebas sin detalles no se pueden implementar

Pruebas de Aceptación/Usuario

- Están directamente relacionadas con los CA
- Describimos lo que vamos a probar
- Probar ---- (falla/pasa)
- Tenemos que contemplar situaciones de éxito y fracaso
- Para poder manejar errores o excepciones → FALLAS

Las PU las define el usuario → hay que elegir muy bien las pruebas que se deben hacer y el usuario las define x q es al que más le duele el negocio entonces él sabe con más certeza lo que se debe probar

Definition of ready

Medida de calidad que define el equipo para marcar si una User puede pasar a desarrollarse y solo pasa si cumple con el DOR → de mínima tiene que cumplir con el criterio que definió el equipo

Un posible DOR como criterio es el modelo INVEST:

*INVEST Model

- **Independent** – calendarizables e implementables en cualquier orden
- **Negotiable** – el “qué” no el “cómo”
- **Valuable** – debe tener valor para el cliente
- **Estimatable** – para ayudar al cliente a armar un ranking basado en costos
- **Small** – deben ser “consumidas” en una iteración
- **Testable** – demostrar que fueron implementadas

Pero el equipo a este modelo puede agregarle que otras cosas se necesitan para considerarse listas las US

Gestión de Productos

En ambientes ágiles

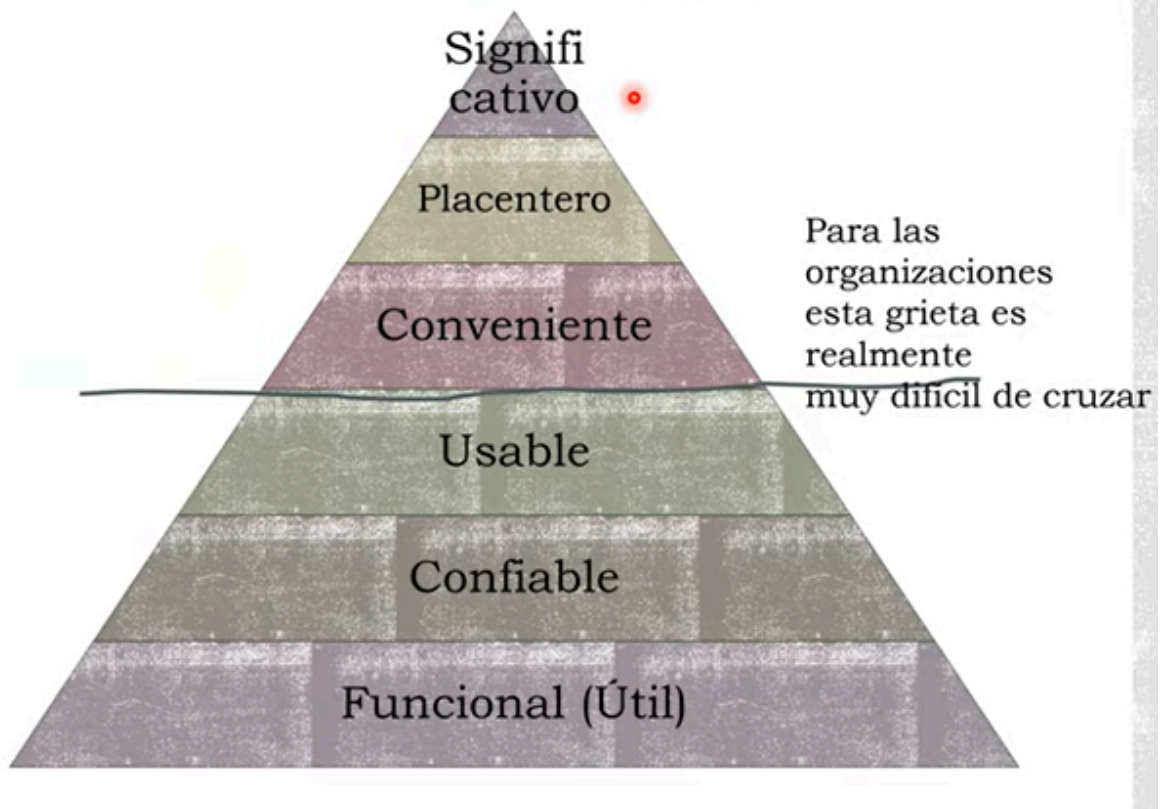
X q creamos productos:

- Para satisfacer a los clientes
- Para tener muchos usuarios logueados
- Para obtener mucho dinero
- Realizar una gran visión, cambiar el mundo

→ Las motivaciones son el disparador

Si se puede desarrollar un producto de manera iterativa e incremental se puede empezar desde una versión mínima y dps de ahí crecer y agrandar

Focalizado en experiencias (gente, actividades, contexto)



Es muy difícil cruzar esa línea → que tanto puedo ser capaz en poner la atención donde debe ir

VALOR DE NEGOCIO → q es lo que el cliente quiere lograr con ese producto.

Como desarrollo un producto con intención de que sea significativo:

Comprender que un producto tiene una hipótesis de valor único:

→ MVP: Producto Mínimo Viable: definición de lo mínimo que necesito para probar mi hipótesis. Lo mínimo probar una hipótesis lo menos costosa y con menos esfuerzo posible. Lo saco al mercado para buscar gente que compruebe o no mi hipótesis o para que se mejore o genere feedbacks de los usuarios.

No necesariamente tiene que ser código, puede ser un video, un powerpoint lo que sea que nos ayude a validar la hipótesis. Y a lo mejor esto nos puede ayudar a cambiar el foco del producto y puede existir una variación o otra versión del producto. En función del feedback vamos viendo como avanzamos con el producto. Puede tener más de una característica principal. Con las US podemos ver cuáles son valiosas para incluir en un MVP y salir a validar la hipótesis. La idea es validar una hipótesis, antes de gastar una fortuna para desarrollar algo que nadie va a querer se hace un MVP. Armar un producto para un mercado que existe y no forzar a los usuarios su uso. Pensar en grande pero empezar con algo chico.

→ MVF: Característica mínima viable, es una versión mini del MVP. Cuando esa característica se valida y sale exitosa se puede seguir desarrollando y se puede convertir en la cabeza de un MVP.

→ MMF: conjunto de características que debe tener mi producto para poder vender/comercializar mi producto . Que se le puede agregar o que se necesita para poder sacarlo al mercado y conseguir venderlo

→ MRF: Características Mínimas del Release. Encaminado hacia un producto maduro que sale al mercado. Cuando el producto esta preparado para salir al mercado hay que definir que características debería tener este release para que cumpla con lo que buscamos del producto.

Relación:

Arrancamos con un MVP es raro que salgamos a validar con una sola característica (MVF) y ese MVP debe salir con un MMP tiene que comercializarse y una vez que salio al mercado y es exitosos ese primer MVP que saque se convierte en mi primer release MMR y mientras sea el producto exitoso y este en el mercado puedo necesitar nuevos releases osea hay que trabajar mas sobre el producto. Para no quedarte fuera del mercado hay que empezar con algo lo mas rápido posible.

FOCO → MVP

- Tiene el valor suficiente para que las personas este dispuestas a usarlo o comprarlo inicialmente
- demuestra suficiente beneficio futuro para retener a los primeros usuarios
- proporciona un ciclo de retroalimentación para guiar el desarrollo futuro

Feedback rápido a un costo razonable, es el producto o no para comercializar ?

No confundir un MVP o un MMP

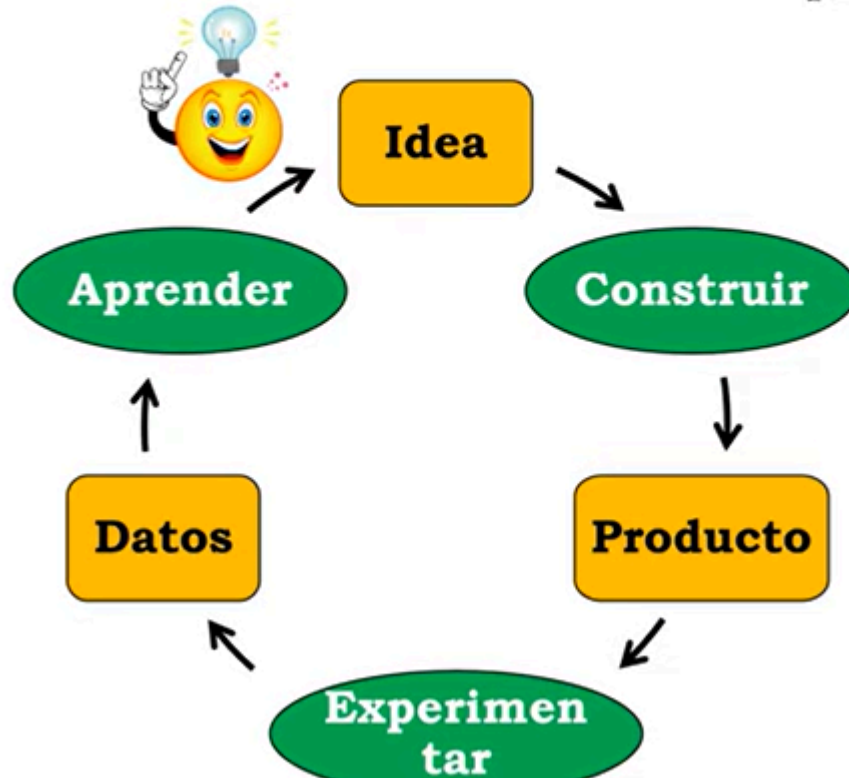
Valor vs Desperdicio

Nos motiva generar un valor de negocio si no otorga ningún valor de negocio es desperdicio

Lean nos dice → que lo que no se usa y no genera valor es desperdicio y hay que eliminar el desperdicio

Lean Start-up → ver

Build-Experiment-Learn Feedback Loop



ciclo donde se encajaría el MVP experimentar

El MVP puede moverse entre la línea de ser código ejecutable un video, etc no si es si es un producto funcionando

La fase construir: MVP

→ Ante la duda quedarse con lo más sencillo, lo más simple y lo más barato

Dilemas a resolver:

Audacia cero: cuando no tienes nada para perder más poder arriesgar, cuando no se tiene nada es, así fácil recaudar el dinero y tener ese incentivo extra

Motivadores:

“Saltos de fe” → se asume que su idea o el problema que quiero resolver es un producto viable

Hipótesis de valor: prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo

- Métrica: tasa de retención

Hipótesis de crecimiento: Prueba como nuevos clientes descubrirán el producto

- Métrica: tasa de referencia o Net Promoter Score (NPS)

Preparar un MVP:

1. Encontrar un Nicho
2. Crear un Roadmap Realista
3. Investigar la competencia
4. MVP
5. Testear las suposición
6. Asegurarse que el MVP resuelve e problema correcto
7. Focalizar en las funcionalidades principales

Ejercicio:

1. Tener el backlog completo
2. Definir una hipótesis
3. Elegir las mínimas funcionalidades que voy a incluir en mi MVP

- Identificar los roles y historias por roles

Tiene que cumplir el objetivo del producto.

Estimaciones de software:

- Tradicional que estimamos:
 - Tamaño
 - Esfuerzo → horas hombre lineales
 - Calendario → cuando
 - Costo
 - Recursos críticos → cosas raras que nos hacen falta fuera de lo normal

Si las estimaciones se utilizan como compromisos son muy peligrosos perjudiciales para cualquier organización

Story points → medida relativa

Se mide con la comparación → tenemos una canónica que usamos para comparar

No es una medida basada en el tiempo

Tamaño ? → tenemos en cuenta estas características para asignarle la cantidad de story points

- Complejidad
- Esfuerzo
- Incertidumbre → que tanta informacion tengo de la implementación o del negocio

La complejidad no es lo mismo que el esfuerzo:

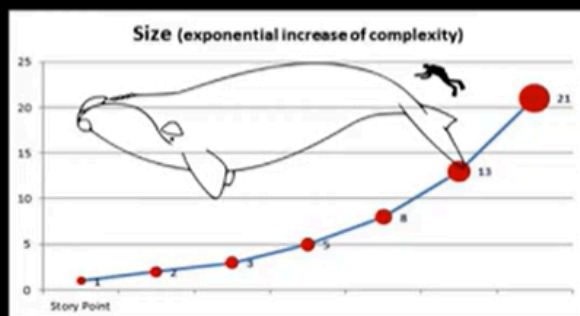
→ Usamos fibonacci para asignarle story points

- Tamaño por números: 1 a 10
- Talles de remeras: S, M, L, XL, XXL
- Serie 2^n : 1, 2, 4, 8, 16, 32, 64, etc.
- Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

Una vez elegida la escala no se cambia! Si se cambia cambiamos el metro patrón

Y qué es un Story Point? (una de muchas definiciones)

- Es una unidad de medida específica (del equipo) de, complejidad, riesgo y esfuerzo, es lo que “el kilo” a la unidad de nuestro sistema de medición de peso
- Story point da idea del “peso” de cada story y decide cuan grande (compleja) es
- La complejidad de una
 - feature/story tiende a
 - incrementarse exponencialmente.



Dos US pueden tener el mismo SP pero no necesariamente tener la misma complejidad, esfuerzo e incertidumbre

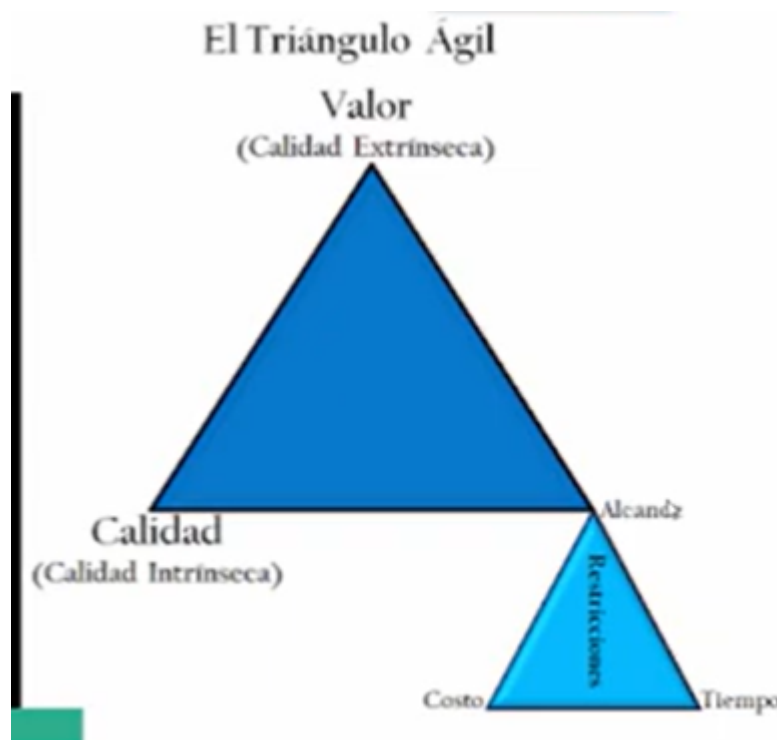
- Poker estimation → combina la opinión de los expertos con la desagregación y analogía

La persona experta para hacer una US es la que va a hacerla y lo tenemos en cuenta cuando asignamos los SP

US canónica → Lo mas simple posible, incertidumbre casi nula y con un esfuerzo minimo;

cada vez que hacemos una estimación comparamos esa US con la canónica para estimar y vemos cuanta mas incertidumbre, esfuerzo y complejidad tiene.

Triangulo de hierro:



Resumen Completo del Paper “No hay balas de Plata”

Idea central

- El desarrollo de software se asemeja a un “hombre lobo”: comienza simple pero puede volverse un monstruo de costos, retrasos y errores.
 - Muchos esperan una “bala de plata” (un avance revolucionario) que resuelva estos problemas.
 - **Tesis de Brooks:** *no existe ni existirá una bala de plata* que produzca una mejora de un orden de magnitud en productividad, fiabilidad o simplicidad.
-

Dificultades esenciales vs. accidentales

Brooks retoma a Aristóteles y distingue:

1. Dificultades esenciales (propias de la naturaleza del software):

- **Complejidad:**
 - El software es más complejo que otras construcciones humanas.
 - No hay partes repetitivas como en edificios o autos; todo es diferente.
 - La complejidad crece de forma no lineal al aumentar el tamaño → dificulta comunicación, pruebas, mantenimiento y seguridad.
- **Conformidad:**
 - El software debe adaptarse a interfaces, leyes, organizaciones, hardware.
 - La mayor parte de la complejidad es arbitraria, impuesta desde fuera.
- **Variabilidad:**
 - El software cambia constantemente por nuevas funciones, nuevos usos y nuevos entornos de hardware.

- Todo software exitoso **evoluciona**.
- **Invisibilidad:**
 - El software no es espacial ni geométrico, no puede representarse en un único diagrama.
 - Los diagramas capturan solo una parte (flujo de datos, control, dependencias), nunca el todo.
 - Esto dificulta el diseño, la comunicación y el control conceptual.

👉 Estas dificultades **no pueden eliminarse**.

2. Dificultades accidentales (derivadas de herramientas/tecnologías limitadas):

- Errores de sintaxis.
- Lenguajes de bajo nivel.
- Lentos ciclos de compilación y ejecución.
- Falta de entornos integrados.

👉 Estas sí se pueden mitigar con avances técnicos.

Avances históricos que resolvieron problemas accidentales

- **Lenguajes de alto nivel:**
 - Aumentaron ×5 la productividad y mejoraron fiabilidad.
 - Al abstraer operaciones, tipos y estructuras, eliminaron un nivel de complejidad innecesaria.
- **Tiempo compartido:**

- Permitted feedback inmediato en vez de esperar horas o días en batch processing.
 - Mejoró la comprensión global de sistemas complejos.
 - **Entornos de desarrollo integrados (Unix, Interlisp):**
 - Simplificaron el uso de librerías, formatos unificados y herramientas compartidas.
-

Supuestas “balas de plata” en los 80s (y por qué no lo son)

1. Lenguaje Ada

- Mejora modularidad, abstracción de datos y estructura jerárquica.
- Pero sigue siendo “otro lenguaje de alto nivel”; no transforma la esencia.

2. Programación orientada a objetos (POO)

- Introduce abstracción de datos y jerarquías de clases.
- Permite expresar mejor el diseño y eliminar sintaxis innecesaria.
- Aun así, no elimina la complejidad esencial → solo reduce problemas accidentales.

3. Inteligencia Artificial

- Brooks distingue:
 - **IA-1:** resolver problemas que antes solo humanos podían.
 - **IA-2:** heurísticas y sistemas basados en reglas.
- Aunque útil en reconocimiento de voz o imágenes, no cambia la dificultad de especificar qué se quiere en software.
- La expresión no es el problema: el verdadero reto es **decidir qué hacer**.

4. Sistemas expertos

- Se pensó aplicarlos a pruebas, diagnósticos y diseño de software.
- Su valor radica en capturar y difundir la experiencia de grandes programadores.
- Pero dependen de disponer de expertos y bases de reglas extensas → limitados.

5. Programación automática

- Se soñó con que la computadora genere el programa desde la especificación.
- Funciona en casos limitados (ej.: ordenamiento, ecuaciones diferenciales), pero no es generalizable al software común.

6. Programación gráfica/visual

- Flujogramas y diagramas gráficos se probaron, pero:
 - Pantallas eran demasiado pequeñas.
 - Los gráficos solo muestran una dimensión del software, no toda su complejidad.
- Brooks lo considera un callejón sin salida.

7. Verificación formal de programas

- Muy útil en software crítico (ej. kernels seguros).
- Pero no ahorra trabajo → verificar es costoso y no asegura que las especificaciones sean correctas.

8. Mejores entornos y estaciones de trabajo

- Aceleran edición, compilación y gestión de detalles.
- Mejoras marginales, no revolucionarias.



Estrategias prometedoras (ataques a la esencia)

1. Comprar en vez de construir software

- Paquetes de software generalistas (nominas, contabilidad, hojas de cálculo).
- El mercado masivo reduce costos por usuario y aumenta productividad.
- Hoy lo vemos en **open source, SaaS, frameworks**.

2. Refinamiento de requisitos + prototipado rápido

- El mayor problema es definir qué se quiere.
- El cliente no sabe lo que necesita hasta ver algo en funcionamiento.
- Prototipos → permiten iterar, descubrir errores en requisitos y mejorar usabilidad.

3. Desarrollo incremental (software que “crece”)

- Harlan Mills propuso “hacer crecer software paso a paso”, no escribirlo de una vez.
- Empieza con un esqueleto que funciona y se completa gradualmente.
- Beneficios: prototipos tempranos, mejor moral del equipo, trazabilidad y facilidad para corregir.

4. Grandes diseñadores

- La diferencia entre un buen diseñador y uno excelente puede ser ×10 en productividad y calidad.
- Ejemplos: Unix, Pascal, Smalltalk (creados por pocos diseñadores brillantes).
- Brooks propone:
 - Identificar talentos temprano.
 - Darles mentores y planes de carrera.
 - Incentivarlos y apoyarlos tanto como a los buenos gestores.

Conclusiones de Brooks

- **No hay balas de plata** → el software seguirá siendo difícil porque su complejidad es esencial.
 - El progreso debe ser **incremental, disciplinado y persistente**.
 - Los caminos más prometedores son:
 - Reutilización y compra de software.
 - Prototipado rápido e iteración de requisitos.
 - Desarrollo incremental.
 - Formación y apoyo a grandes diseñadores.
-

✨ Frases clave para recordar

- *“La complejidad del software es esencial, no accidental.”*
- *“No hay una bala de plata para la ingeniería del software.”*
- *“La parte más difícil de hacer software es decidir qué hacer.”*
- *“El secreto es que el software debe crecer, no construirse de una vez.”*

Ejercicios TIPO PARCIAL: